UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE
CORSO DI STUDIO TRIENNALE IN INFORMATICA - F004

Sviluppo di un sistema embedded per il controllo della temperatura in una camera di collaudo

Relatore:
Prof. Carlo Dossi

Tesi di Laurea di:
Mattia Papaccioli - 747053

Tutor Aziendale:
Edoardo Scaglia

Azienda ospitante:
AMEL SRL

Anno Accademico:
2025/2026

Indice

1 Introduzione **1**
1.1 Abstract 1
1.1.1 Architettura del sistema di controllo 1
1.1.2 Interfacciamento hardware e comunicazione 2
1.1.3 Interfaccia utente e sistema remoto 2
1.2 Struttura del codice sorgente 3

2 Moduli del sistema **4**
2.1 Il modulo `common-control` 4
2.1.1 Meccanismi di comunicazione inter-modulare 4
2.1.2 La libreria `logging.h` 5
2.1.3 Template Bash con preprocessore C 5
2.1.4 Procedura di esecuzione del sistema 6
2.1.5 Interfaccia web tramite CGI 6
2.2 Il modulo `temp-control` 7
2.2.1 Schermata principale 7
2.2.2 Gestione degli eventi di interazione 8
2.2.3 Backend per l'input/output 9
2.2.4 Configurazione della cross-compilazione 9
2.2.5 Strategia di branching per lo sviluppo 10
2.3 Il modulo `pid-control` 10
2.3.1 Acquisizione dati dai sensori di temperatura 10
2.3.2 Comunicazione MODBUS RTU 13
2.3.3 Algoritmo di controllo PID 13
2.3.4 Pannello di amministrazione 14
2.3.5 Sistema di logging e monitoraggio 14

3 Sistema operativo embedded Linux **16**
3.1 Architettura hardware del sistema 16
3.2 Realizzazione del sistema operativo 17
3.3 Personalizzazione di Buildroot 17
3.3.1 Il pacchetto `amel-common-control` 18
3.3.2 Il pacchetto `amel-temp-control` 18
3.3.3 Il pacchetto `amel-pid` 19
3.3.4 Configurazione dei servizi di rete 20
3.3.5 Supporto per la porta seriale RS-232 20
3.3.6 Sincronizzazione temporale con NTP 21
3.4 Personalizzazione delle patch per Barebox e Linux 21

4 Installazione del sistema **22**
4.1 Configurazione della comunicazione con l'inverter 22
4.2 Taratura dei parametri del controllore PID 22
4.2.1 Fase preparatoria 23

4.2.2 Identificazione del parametro proporzionale 23
4.2.3 Integrazione della componente integrale 24
4.3 Risoluzione dei problemi riscontrati 25
4.3.1 Gestione della concorrenza nel framework LVGL 25
4.3.2 Gestione delle risorse nella comunicazione MODBUS 26

5 Manuale utente **30**
5.1 Compilazione del sistema operativo tramite Buildroot 30
5.1.1 Procedura di clonazione del repository 30
5.1.2 Processo di compilazione 31
5.2 Preparazione della scheda di memoria SD 31
5.2.1 Procedura di scrittura dell'immagine 31
5.3 Configurazione del dispositivo embedded 32
5.3.1 Avvio del sistema 32
5.3.2 Modalità di connessione al dispositivo 32
5.4 Inizializzazione e avvio del sistema 36
5.4.1 Configurazione dell'ambiente 36
5.4.2 Calibrazione del touchscreen 37
5.4.3 Script di gestione del sistema 37
5.4.4 Monitoraggio e logging 38

6 Conclusione **39**
6.1 L'importanza del testing integrato sul campo 39
6.2 Competenze acquisite e riflessioni professionali 40
6.2.1 Strumenti di sviluppo collaborativo 40
6.2.2 Architetture embedded 40
6.2.3 Interdisciplinarità ingegneristica 40
6.2.4 Applicazione di protocolli industriali 40
6.3 Ringraziamenti 41

A Riferimenti **42**

1

Introduzione

1.1 Abstract

Il presente elaborato descrive la progettazione e lo sviluppo di un sistema embedded per il controllo della temperatura all'interno di una camera di collaudo, situata presso AMEL s.r.l. Tale ambiente viene utilizzato per l'esecuzione di test su carichi resistivi, i quali generano una significativa quantità di calore durante il funzionamento. Per garantire la correttezza e la ripetibilità delle prove sperimentali, risulta dunque necessario mantenere la temperatura ambientale entro limiti predefiniti, particolarmente durante i periodi estivi.

1.1.1 Architettura del sistema di controllo

Il sistema proposto implementa un circuito di regolazione automatica basato su un controllore PID (Proporzionale-Integrativo-Derivativo), il quale consente di modulare la frequenza di rotazione di una ventola di raffreddamento. Il funzionamento del sistema può essere descritto attraverso le seguenti fasi operative:

1. **Rilevazione della temperatura**: un sensore dedicato misura la temperatura ambiente presente all'interno della camera di collaudo;
2. **Calcolo dell'errore**: la temperatura rilevata viene confrontata con il valore di riferimento (setpoint) per determinare lo scostamento istantaneo;
3. **Elaborazione del segnale di controllo**: l'algoritmo PID elabora l'errore calcolando l'azione proporzionale, basata sull'errore attuale, e l'azione integrativa, derivante dall'accumulo degli errori nel tempo. Il risultato di tale elaborazione consiste in un segnale di tensione da inviare all'inverter;
4. **Attuazione del comando**: l'inverter riceve il segnale di tensione e lo converte nella frequenza di alimentazione della ventola;

5. **Retroazione continua**: il sistema opera in modalità di retroazione

negativa, applicando ciclicamente le fasi precedenti per mantenere la temperatura il più possibile vicina al valore target.

1.1.2 Interfacciamento hardware e comunicazione

La comunicazione bidirezionale tra il sistema embedded e l'inverter di frequenza avviene attraverso il protocollo di comunicazione industriale MODBUS RTU [1]. Tale protocollo garantisce la trasmissione affidabile dei dati su bus seriale, risultando particolarmente adatto per applicazioni di controllo in tempo reale.

1.1.3 Interfaccia utente e sistema remoto

Per facilitare l'interazione con il sistema, è stata realizzata un'interfaccia grafica utente (GUI) che consente di visualizzare lo stato attuale del sistema e di modificare i parametri operativi, inclusa la temperatura target desiderata. L'interfaccia viene presentata su un display touchscreen integrato nel pannello di controllo, garantendo immediatezza d'uso anche per operatori senza specifica formazione tecnica. La libreria grafica LVGL [2] è stata adottata per lo sviluppo dell'interfaccia, in virtù della sua efficienza nell'ambito dei sistemi embedded a risorse limitate.

Il sistema embedded è inoltre dotato di connettività Ethernet, che ne consente l'integrazione all'interno della rete aziendale. Tramite un'architettura client-server basata su web server, gli operatori autorizzati possono accedere alle funzionalità di controllo anche da remoto, modificando la temperatura target senza necessità di presenza fisica presso la camera di collaudo. La comunicazione tra il sistema embedded e il web server avviene attraverso un meccanismo di scrittura e lettura su file condivisi.

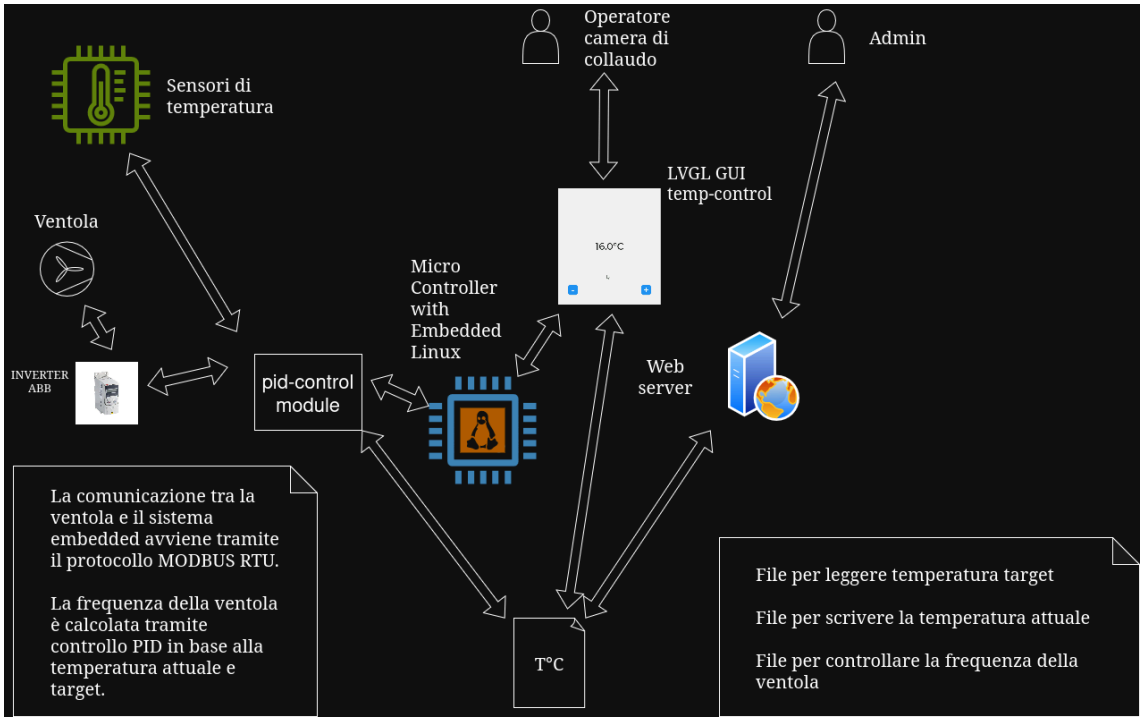

Figura 1 — Diagramma d'insieme del sistema di controllo temperatura

1.2 Struttura del codice sorgente

Il software sviluppato per il sistema embedded è organizzato secondo un'architettura modulare, suddivisa in tre componenti principali:

- **`common-control`**: modulo di supporto contenente funzioni di utilità

condivise, definizioni di costanti e strutture dati utilizzate per la comunicazione tra i livelli applicativi;

- **`temp-control`**: modulo dedicato alla gestione dell'interfaccia grafica

e all'elaborazione degli input utente, implementato mediante la libreria LVGL;

- **`pid-control`**: modulo centrale responsabile dell'implementazione

dell'algoritmo PID, dell'acquisizione dati dal sensore di temperatura e della comunicazione MODBUS RTU con l'inverter [1].

La suddivisione in moduli indipendenti è stata adottata per favorire la manutenibilità del codice e la possibilità di estensione futura del sistema. Tale approccio consente infatti di sostituire o modificare singoli componenti senza necessità di interventi strutturali sull'intero progetto, offrendo vantaggi significativi rispetto a una soluzione monolitica.

2

Moduli del sistema

Il sistema di controllo della temperatura è stato progettato seguendo un'architettura modulare che permette una chiara separazione delle responsabilità tra i diversi componenti software. Tale approccio facilita la manutenibilità del codice e consente di aggiornare singoli moduli senza compromettere il funzionamento dell'intero sistema. I due moduli principali sono `temp-control` e `pid-control`, coordinati dal modulo di supporto `common-control`.

2.1 Il modulo `common-control`

Il modulo `common-control` rappresenta il nucleo di supporto dell'intero sistema. Esso svolge tre funzioni fondamentali: fornisce funzioni di utilità condivise tra gli altri moduli, definisce le configurazioni globali attraverso file header, e gestisce gli script per l'inizializzazione e l'avvio del sistema.

La struttura delle directory del sistema è organizzata come segue: tutti gli script eseguibili e i file di configurazione risiedono nella directory di sistema `/opt/amel/`. Questa scelta garantisce un'ubicazione standardizzata e protetta per i componenti del sistema di controllo.

2.1.1 Meccanismi di comunicazione inter-modulare

Per garantire lo scambio di dati tra i moduli `temp-control` e `pid-control` è stato necessario implementare un sistema di comunicazione che combina l'uso dei segnali Unix con operazioni di lettura e scrittura su file.

Il flusso di comunicazione avviene secondo questa sequenza:

1. L'operatore modifica la temperatura target attraverso l'interfaccia grafica

sul display LCD

2. Il modulo `temp-control` scrive il nuovo valore nel file

`/opt/amel/target-temperature`

3. Il processo `pid-control`, eseguendo ciclicamente la lettura dei

sensori di temperatura DS18B20 [3], memorizza le misurazioni nei file `/opt/amel/current-temperature/sX`, dove `X` indica il numero identificativo del sensore sul bus

4. Dopo ogni scrittura, `pid-control` invia un segnale Unix al processo

`temp-control`

5. Il modulo `temp-control`, alla ricezione del segnale, legge i file

aggiornati e aggiorna l'interfaccia grafica

Questo approccio ibrido consente una comunicazione affidabile anche in presenza di operazioni bloccanti, poiché i segnali notificano immediatamente la disponibilità di nuovi dati.

2.1.2 La libreria `logging.h`

Per garantire una gestione ordinata dei messaggi di sistema è stata sviluppata una libreria di logging in linguaggio C. Questa libreria supporta la formattazione delle stringhe e la classificazione dei messaggi secondo sei livelli di gravità: `TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR` e `FATAL`.

Le caratteristiche principali della libreria sono:

- Filtro selettivo dei messaggi in base al livello di gravità configurato
- Possibilità di scrittura su file di sistema log o su console
- Meccanismo di sincronizzazione tramite mutex per prevenire sovrapposizioni

di output durante accessi concorrenti

- Configurazione della precisione temporale, con risoluzione fino al

microsecondo, utile per il debugging della regolarità del controllo PID

L'utilizzo dei mutex è fondamentale per garantire l'integrità dei messaggi di log quando più processi tentano di scrivere contemporaneamente sullo stesso file di output.

2.1.3 Template Bash con preprocessore C

Per eliminare la duplicazione delle costanti di configurazione è stato adottato un approccio innovativo che sfrutta il preprocessore del linguaggio C per la generazione di script Bash.

Il procedimento si articola nelle seguenti fasi:

1. Le configurazioni globali vengono definite nel file header

`include/config.h`

2. Queste definizioni vengono utilizzate come input per template di script

3. Gli script generati sono `init.sh` per l'inizializzazione del sistema e

`run.sh` per l'avvio dei moduli

4. Il preprocessore C viene invocato attraverso il compilatore GCC con la

flag `-E`, che esegue solamente la fase di preprocessing senza compilazione, assemblaggio o linking

Questa metodologia presenta il vantaggio significativo di mantenere tutte le costanti di configurazione in un unico punto: modificando il file header `config.h`, tutti gli script vengono automaticamente aggiornati con i valori corretti, eliminando rischi di inconsistenza.

2.1.4 Procedura di esecuzione del sistema

L'avvio del sistema embedded richiede l'esecuzione sequenziale di script dedicati, ciascuno con una funzione specifica.

Al primo avvio del dispositivo è necessario eseguire lo script `/opt/amel/init.sh`. Questo script svolge le seguenti operazioni:

- Creazione delle directory necessarie per la gestione dei sensori di

temperatura

- Configurazione del controllo degli accessi ai file di sistema

Per l'avvio operativo del sistema viene utilizzato lo script `/opt/amel/run.sh`, che orchestra l'esecuzione dei moduli principali:

1. Viene avviato per primo il processo principale di `temp-control`, che si

mette in attesa del segnale di sincronizzazione

2. Successivamente viene lanciato `pid-control`, il quale inizializza il bus

One-Wire e rileva il numero di sensori connessi

3. Il numero di sensori rilevato viene memorizzato nel file

`/opt/amel/number-sensors`

4. Una volta completata l'inizializzazione, `pid-control` invia il segnale

di sincronizzazione a `temp-control`

5. Entrambi i processi entrano in esecuzione normale, scambiandosi

aggiornamenti sulla temperatura target e corrente attraverso segnali Unix generati con il comando `kill`

2.1.5 Interfaccia web tramite CGI

Per consentire il monitoraggio e il controllo remoto del sistema è stata implementata un'interfaccia web basata su CGI (Common Gateway Interface). Questa soluzione permette di accedere alle funzionalità del sistema attraverso un browser web standard.

Le funzionalità dell'interfaccia web includono:
- Visualizzazione in tempo reale della temperatura attuale rilevata dai sensori
- Modifica della temperatura target mediante pulsanti di incremento e

decremento
- Hosting diretto sul dispositivo embedded, senza necessità di infrastrutture

esterne

La comunicazione tra il server web e i moduli del sistema avviene attraverso le stesse primitive di file system utilizzate per la comunicazione inter-modulare, garantendo coerenza nell'accesso ai dati.

2.2 Il modulo `temp-control`

Il modulo `temp-control` gestisce l'interfaccia utente del sistema attraverso un display touchscreen LCD. Questo modulo consente all'operatore di visualizzare i parametri di funzionamento e di impostare la temperatura desiderata nella camera di collaudo.

L'interfaccia grafica è stata sviluppata utilizzando la libreria LVGL [2] (Light and Versatile Graphics Library), scelta per le sue caratteristiche di leggerezza e adattabilità ai sistemi embedded. Il punto di partenza è stato un template [4] che fornisce il porting della libreria su sistemi operativi Linux, messo a disposizione dagli sviluppatori stessi della libreria.

2.2.1 Schermata principale

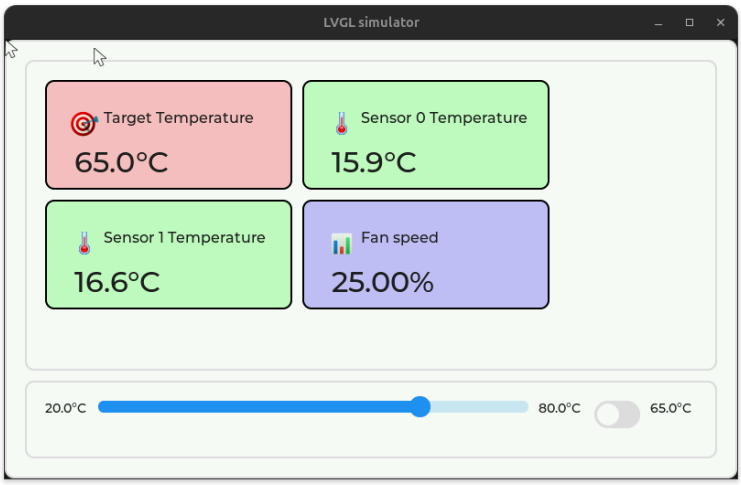

Figura 2 — Interfaccia grafica per il controllo della temperatura

L'interfaccia principale presenta le seguenti informazioni:
- La temperatura target impostata dall'operatore
- Le temperature attuali rilevate dai sensori connessi al bus One-Wire
- Due pulsanti per l'incremento e il decremento della temperatura target

La disposizione degli elementi è stata progettata per garantire la massima leggibilità anche in condizioni di scarsa illuminazione della camera di collaudo.

2.2.2 Gestione degli eventi di interazione

```c
void set_target_temperature(float t)
{
    LOG_DEBUG("debug callback -> %.1f\n", target_temperature);
    target_temperature += t;
    lv_label_set_text_fmt(target_temperature_label,
    target_temperature_format, target_temperature);
    write_float_to_file(TARGET_TEMPERATURE_FILE, target_temperature);
    kill(PID_control_PID, SIGUSR1);
}

static void increment_temperature(lv_event_t * e)
{
    if(lv_event_get_code(e) != LV_EVENT_CLICKED) {
        return;
    }
    set_target_temperature(1);
}

static void decrement_temperature(lv_event_t * e)
{
    if(lv_event_get_code(e) != LV_EVENT_CLICKED) {
        return;
    }
    set_target_temperature(-1);
}
```

Codice 1 — Funzioni di callback per la gestione dei pulsanti di incremento e decremento della temperatura target

Quando l'operatore preme uno dei pulsanti per modificare la temperatura target, il sistema esegue la seguente sequenza di operazioni:

1. Viene invocata la funzione di callback corrispondente all'evento di click
2. La funzione `set_target_temperature` aggiorna il valore della variabile

interna

3. L'etichetta grafica (label) che visualizza la temperatura target viene

aggiornata

4. Il nuovo valore viene scritto nel file di interfaccia per la comunicazione

con il modulo PID

5. Viene inviato un segnale Unix al processo `pid-control` per notificare

il cambiamento

2.2.3 Backend per l'input/output

Il modulo utilizza due backend fondamentali per la gestione dell'input/output:
- Libevdev per la gestione degli eventi di input dal touchscreen
- Il framebuffer device per la visualizzazione grafica

Libevdev [5] è una libreria di astrazione per i dispositivi di input del kernel Linux. Essa riceve gli eventi di tocco dal touchscreen e li traduce in un formato comprensibile per l'interfaccia grafica LVGL, gestendo automaticamente la calibrazione e la mappatura delle coordinate.

Il framebuffer device è rappresentato dal file speciale `/dev/fb0`. Questo file contiene una rappresentazione della memoria video: scrivendo in questa memoria, la libreria LVGL aggiorna direttamente i pixel visualizzati sullo schermo LCD. Questo approccio garantisce bassa latenza e ridotto overhead computazionale.

2.2.4 Configurazione della cross-compilazione

La compilazione dell'applicazione richiede una toolchain specifica per l'architettura ARM del dispositivo target. Nel presente progetto si è fatto affidamento sul compilatore GCC e sulle librerie fornite dal sistema di build Buildroot.

Configurazione della toolchain

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(tools ~/buildroot/output/host/bin/arm-buildroot-linux-gnueabihf-)
set(CMAKE_C_COMPILER ${tools}gcc)
set(CMAKE_CXX_COMPILER ${tools}g++)

set(EVDEV_INCLUDE_DIRS ~/buildroot/output/staging/usr/include/libevdev/)
set(EVDEV_LIBRARIES ~/buildroot/output/staging/usr/lib/libevdev.so)

set(BUILD_SHARED_LIBS ON)
```

Codice 2 — Configurazione della toolchain per la cross-compilazione (cross_compile_setup.cmake)

Il file di configurazione CMake definisce i seguenti parametri:
- Il sistema operativo di destinazione (Linux) e l'architettura del processore

(ARM)
- Il percorso del compilatore GCC e del compilatore C++ specifici per la

toolchain
- Le directory di inclusione e le librerie per libevdev
- La modalità di compilazione delle librerie condivise

La generazione dei makefile avviene tramite il comando: `cmake -DCMAKE_TOOLCHAIN_FILE=./cross_compile_setup.cmake -B build -S .`

Successivamente, la compilazione vera e propria viene eseguita con: `make -C build -j` [6]

La libreria LVGL viene compilata come libreria condivisa (shared library), mentre l'applicazione principale viene generata come eseguibile standalone.

2.2.5 Strategia di branching per lo sviluppo

Per organizzare efficientemente il codice sorgente durante le diverse fasi del progetto è stata adottata una strategia di branching basata su due repository principali: una dedicata allo sviluppo su PC e una destinata al dispositivo embedded.

Le due repository presentano una differenza sostanziale nel file `lv_conf.h`, che contiene la configurazione specifica della libreria LVGL:

- Branch di sviluppo: utilizza il backend `x11` per la visualizzazione su

desktop Linux e include controlli di coerenza (sanity check) utili durante il debugging, ma penalizzanti in termini di performance

- Branch target: disabilita i sanity check e utilizza il backend `/dev/fb0`

per l'accesso diretto al framebuffer del dispositivo embedded

Per preservare le diverse configurazioni durante le operazioni di merge è stato configurato il file `.gitattributes` con la direttiva `lv_conf.h merge=ours`. Questa impostazione istruisce Git a mantenere sempre la versione del file presente nella branch corrente durante le operazioni di fusione, prevenendo sovrascritture accidentali delle configurazioni specifiche.

2.3 Il modulo `pid-control`

Il modulo `pid-control` rappresenta il nucleo computazionale del sistema, responsabile dell'acquisizione dei dati dai sensori, dell'elaborazione del controllo PID e della comunicazione con l'attuatore (l'inverter della ventola).

2.3.1 Acquisizione dati dai sensori di temperatura

I sensori di temperatura utilizzati sono due dispositivi DS18B20 [3], collegati in configurazione parallela su un bus 1-Wire. Questa topologia consente di collegare multiple unità di misura utilizzando un'unica linea dati.

Il microcontrollore agisce come master sul bus, interrogando periodicamente i sensori per ottenere le misurazioni di temperatura. L'eseguibile PID esegue ciclicamente le seguenti operazioni:

1. Lettura della temperatura dai sensori
2. Calcolo dell'output del controllore PID in base alla temperatura misurata

e al setpoint desiderato

3. Aggiornamento del comando per l'inverter

La procedura di inizializzazione dei sensori avviene in tre fasi distinte:

1. Conteggio del numero di sensori presenti sul bus
2. Allocazione della memoria necessaria per memorizzare gli identificatori

univoci (UUID) dei sensori

3. Lettura e memorizzazione degli UUID in strutture dati dedicate

Questo approccio a due fasi è preferibile rispetto a una lettura continua del bus, in quanto evita chiamate ripetute alla funzione `DS18X20_find_sensor`, riducendo l'overhead computazionale. La memoria allocata per gli UUID viene mantenuta per tutta la durata dell'esecuzione del programma, rappresentando un trade-off accettabile tra utilizzo di memoria e efficienza computazionale.

```c
typedef struct
{
    char id[16];
    int16_t temperature;
    uint8_t uint_id[OW_ROMCODE_SIZE];
} sensor;

void init_onewire_sensors()
{
    uint8_t diff = OW_SEARCH_FIRST;

    while (diff != OW_LAST_DEVICE)
    {
        sensors_count++;
        DS18X20_find_sensor(&diff, id);
    }

    write_char_to_file(NUMBER_OF_SENSORS_FILE, sensors_count);
    LOG_INFO("Found %d sensors on the 1-wire bus", sensors_count);

    sensors = malloc(sizeof(sensor) * sensors_count);

    diff = OW_SEARCH_FIRST;
    int i = 0;
    while (diff != OW_LAST_DEVICE)
    {
        DS18X20_find_sensor(&diff, id);
        sensor s;
        for (int i = 0; i < OW_ROMCODE_SIZE; i++)
        {
            s.uint_id[i] = id[i];
        }
        sprintf(s.id, "%02hx%02hx%02hx%02hx%02hx%02hx%02hx%02hx",
        id[0], id[1],
              id[2], id[3], id[4], id[5], id[6], id[7]);
        s.temperature = 0;
        sprintf(s.file, CURRENT_TEMPERATURE_FILE "/s%d", i);
        LOG_INFO("sensor %d id %s", i, s.id);
        sensors[i] = s;
        i++;
    }
}
```

Codice 3 — `pid/src/main.c`

Controllo di integrità CRC-8

Per garantire l'affidabilità dei dati trasmessi dal sensore al microcontrollore è stato implementato un meccanismo di controllo di integrità basato sull'algoritmo CRC-8 (Cyclic Redundancy Check a 8 bit).

L'algoritmo utilizza il polinomio 0x18, corrispondente alla rappresentazione matematica $x^8 + x^5 + x^4 + x^0$. Questo polinomio generatore viene applicato ai dati dello scratchpad del sensore per calcolare un valore di checksum.

Il processo di verifica si svolge come segue:

1. Il sensore trasmette i dati di temperatura insieme al valore CRC calcolato
2. Il microcontrollore ricalcola il CRC sui dati ricevuti
3. Se il valore calcolato corrisponde a quello ricevuto, i dati sono

considerati validi

4. In caso di discrepanza, la lettura viene scartata e può essere ritentata

nel ciclo successivo

Questo meccanismo è fondamentale per prevenire azioni di controllo basate su dati corrotti, che potrebbero compromettere la stabilità del sistema.

2.3.2 Comunicazione MODBUS RTU

Per il comando dell'inverter che regola la velocità della ventola di raffreddamento è stato adottato il protocollo di comunicazione MODBUS RTU (Remote Terminal Unit). Questo protocollo seriale è uno standard industriale per la comunicazione tra dispositivi elettronici.

L'implementazione del protocollo si avvale della libreria open source `libmodbus` [1], che fornisce un'interfaccia di alto livello per la gestione delle comunicazioni MODBUS. La libreria gestisce automaticamente:

- La codifica e decodifica dei frame di comunicazione
- Il calcolo del checksum
- La gestione degli errori di trasmissione
- Il timeout delle risposte

2.3.3 Algoritmo di controllo PID

Il controllore PID (Proporzionale, Integrale, Derivativo) è un sistema di retroazione negativa ampiamente utilizzato nell'industria per il controllo di processi. Il suo principio di funzionamento si basa sulla correzione continua di un errore rispetto a un valore di riferimento desiderato.

Nel contesto del presente sistema, il controllore PID svolge le seguenti funzioni:

1. Acquisisce in ingresso la temperatura misurata dai sensori
2. Confronta il valore misurato con il setpoint di temperatura desiderato
3. Calcola l'errore come differenza tra setpoint e valore misurato
4. Elabora l'errore attraverso tre componenti:
   - Il termine proporzionale, che reagisce all'entità dell'errore istantaneo
   - Il termine integrale, che elimina l'errore residuo nel tempo
   - Il termine derivativo, che anticipa le variazioni future dell'errore
5. Produce in uscita un segnale di comando che viene convertito in tensione

per l'inverter

6. L'inverter regola di conseguenza la frequenza di rotazione della ventola

di raffreddamento

Gestione temporale con monotonic clock

Per garantire la corretta esecuzione dell'algoritmo PID è fondamentale mantenere una frequenza di campionamento costante. La variabilità del periodo di campionamento comprometterebbe l'efficacia dei calcoli integrale e derivativo.

A questo scopo è stato utilizzato il meccanismo dei timer file descriptor fornito dall'header `<sys/timerfd.h>` della Standard C Library. Questa API del kernel Linux consente di:

1. Creare un timer che scade a intervalli regolari e predefiniti
2. Ricevere notifiche di scadenza attraverso un file descriptor
3. Utilizzare un clock monotonico, che non è influenzato da variazioni

dell'ora di sistema

L'utilizzo del monotonic clock garantisce che il periodo di campionamento rimanga costante anche in presenza di aggiustamenti dell'orologio di sistema o di sincronizzazioni NTP.

Gestione delle priorità dello scheduler

Per minimizzare le interferenze durante le operazioni critiche di misurazione e controllo, al processo è stata assegnata la massima priorità di esecuzione disponibile per lo scheduler del kernel Linux. Questa configurazione viene impostata attraverso le funzioni definite nell'header `<sched.h>`.

Le ragioni di questa scelta sono:
- Prevenire l'interruzione del processo di campionamento da parte di processi

a priorità inferiore
- Ridurre la latenza nella risposta agli eventi del timer
- Mantenere la massima regolarità possibile nel periodo di campionamento

L'assegnazione della priorità massima richiede privilegi di amministratore (root), coerentemente con il fatto che il modulo viene eseguito come servizio di sistema.

2.3.4 Pannello di amministrazione

Il modulo include funzionalità per la configurazione e la manutenzione del sistema, accessibili attraverso un'interfaccia dedicata. Queste funzioni consentono di:
- Visualizzare lo stato operativo del sistema
- Modificare i parametri del controllore PID
- Eseguire operazioni diagnostiche sui sensori

2.3.5 Sistema di logging e monitoraggio

Per consentire l'analisi dell'andamento del controllo nel tempo e per facilitare le operazioni di debugging è stata implementata una funzionalità di logging su file in formato CSV (Comma Separated Values).

La procedura di logging si articola nelle seguenti fasi:

1. All'avvio del programma viene generato un file CSV con un nome basato sul timestamp corrente, garantendo l'unicità del file
2. L'intestazione del file contiene i nomi delle colonne: tempo, temperatura target, temperature dei singoli sensori, output del PID
3. Ad ogni ciclo di esecuzione del controllo, i valori correnti vengono aggiunti come nuova riga del file
4. Il file può essere successivamente analizzato mediante strumenti software o fogli di calcolo per valutare le prestazioni del sistema

Questo approccio di logging strutturato consente di:

- Ricostruire la cronologia delle operazioni del sistema
- Analizzare la risposta del controllore a variazioni del setpoint
- Identificare eventuali anomalie nel comportamento dei sensori
- Ottimizzare i parametri del PID mediante analisi dei dati storici

3

Sistema operativo embedded Linux

3.1 Architettura hardware del sistema

Il sistema embedded sviluppato per il controllo della temperatura si basa su un'architettura hardware progettata da AMEL, costituita da due componenti principali:

- **Host-board Ganador (revisione 4)**: rappresenta la scheda portante del sistema e integra le seguenti periferiche:
  - memoria SD, utilizzata come dispositivo di storage principale;
  - interfaccia Ethernet per la connettività di rete;
  - interfaccia seriale per la comunicazione con dispositivi esterni;
  - display touchscreen per l'interazione con l'utente.
- **System-on-Module Vulcano-A5**: costituisce il modulo di elaborazione

centrale e comprende:

- processore Atmel ARM9 AT91SAM9X35 a 400 MHz;
- 128 MB di memoria DDR2 SDRAM;
- 256 MB di memoria NAND Flash;
- interfaccia SODIMM200;
- controller Ethernet MAC 10/100 Mbps;
- due porte USB 2.0 Host e una porta USB 2.0 Host/Device;
- tre interfacce USART;
- controller TFT LCD con supporto TTL/LVDS.

3.2 Realizzazione del sistema operativo

Per la gestione del sistema embedded è stato adottato il sistema operativo Linux [7], noto per la sua flessibilità e per la possibilità di personalizzazione del kernel. Data la limitatezza delle risorse hardware disponibili, è stato necessario procedere alla costruzione di un kernel su misura, includendo esclusivamente i moduli essenziali per il funzionamento del sistema.

Per la compilazione incrociata (cross-compilation) e la creazione del filesystem è stato impiegato Buildroot [8]. Questo strumento consiste in una raccolta di Makefile che automatizzano le seguenti operazioni:

1. installazione e compilazione delle librerie necessarie;
2. gestione delle dipendenze tra i pacchetti;
3. creazione del filesystem completo;
4. generazione di un'immagine pronta per essere trasferita sulla scheda SD

del dispositivo target.

L'integrazione dell'interfaccia grafica sviluppata con la libreria LVGL ha richiesto la creazione di un pacchetto personalizzato all'interno di Buildroot, come descritto nelle sezioni successive.

La sequenza di avvio del sistema segue il seguente flusso:

1. il bootloader del chip attiva AT91bootstrap [9];
2. AT91bootstrap avvia il bootloader secondario Barebox [10];
3. Barebox carica il kernel Linux in memoria e ne avvia l'esecuzione.

3.3 Personalizzazione di Buildroot

L'aggiunta di nuovi pacchetti software a Buildroot richiede la definizione di una struttura specifica all'interno della directory `package`. In particolare, per ciascun pacchetto è necessario creare:

- un file `Config.in`, che descrive le opzioni di configurazione e le

dipendenze del pacchetto;

- un file `.mk`, che contiene le istruzioni per il download, la compilazione

e l'installazione del software.

Questi file consentono a Buildroot di risolvere automaticamente le dipendenze, scaricare i sorgenti e installare il pacchetto nel filesystem del dispositivo target.

3.3.1 Il pacchetto `amel-common-control`

Il pacchetto `amel-common-control` rappresenta una libreria di supporto condivisa tra i vari componenti del sistema. La configurazione di Buildroot per questo pacchetto è riportata nel Codice sorgente 3.1.

```
AMEL_COMMON_CONTROL_VERSION = v0.9
AMEL_COMMON_CONTROL_SITE =
git@git.amelchem.com:mpapaccioli/common-control.git
AMEL_COMMON_CONTROL_SITE_METHOD = git

define AMEL_COMMON_CONTROL_BUILD_CMDS
  $(TARGET_MAKE_ENV) TOOLS=$(TARGET_CROSS) STAGE=$(STAGING_DIR) \
  cmake -S $(@D) -B $(@D)/build -D CMAKE_BUILD_TYPE=Release -D  \
  CMAKE_INSTALL_PREFIX=/usr                      \
  -DCMAKE_TOOLCHAIN_FILE=$(@D)/user_cross_compile_setup.cmake   \
  -DBUILD_SHARED_LIBS=ON
  $(TARGET_MAKE_ENV) cmake --build $(@D)/build
  make -C $(@D) source/init.sh source/run.sh
endef

define AMEL_COMMON_CONTROL_INSTALL_TARGET_CMDS
  $(INSTALL) -d $(TARGET_DIR)/opt/amel
  $(INSTALL) -m 0755 $(@D)/source/init.sh $(TARGET_DIR)/opt/amel/init.sh
  $(INSTALL) -m 0755 $(@D)/source/run.sh $(TARGET_DIR)/opt/amel/run.sh
endef

$(eval $(generic-package))
```

Codice 4 — Configurazione Buildroot per il pacchetto amel-common-control

3.3.2 Il pacchetto `amel-temp-control`

Il pacchetto `amel-temp-control` contiene l'applicazione con interfaccia grafica sviluppata mediante la libreria LVGL. La sua configurazione Buildroot è illustrata nel Codice sorgente 3.2.

```
AMEL_TEMP_CONTROL_VERSION = 44c17c6f2c492f1f3c7d8a6767df390c8d13eb9c
AMEL_TEMP_CONTROL_SITE = git@git.amelchem.com:mpapaccioli/temp-control.git
AMEL_TEMP_CONTROL_SITE_METHOD = git

AMEL_TEMP_CONTROL_DEPENDENCIES = libevdev
AMEL_TEMP_CONTROL_GIT_SUBMODULES = YES

define AMEL_TEMP_CONTROL_BUILD_CMDS
  cmake -DCMAKE_TOOLCHAIN_FILE=$(@D)/user_cross_compile_setup.cmake \
    -B $(@D)/build -S $(@D)
  make -C $(@D)/build -j @cmake
endef

define AMEL_TEMP_CONTROL_INSTALL_TARGET_CMDS
  $(INSTALL) -d $(TARGET_DIR)/opt/amel-temp-control/
  cp $(@D)/build/bin/lvglsim $(TARGET_DIR)/opt/amel-temp-control/main
  cp -r $(@D)/build/lvgl/lib/* $(TARGET_DIR)/usr/lib
endef

$(eval $(generic-package))
```

Codice 5 — Configurazione Buildroot per il pacchetto amel-temp-control

Il processo di compilazione e installazione del pacchetto si articola nelle seguenti fasi:

1. **Clonazione dei sorgenti**: il repository `temp-control`, contenente il

codice sorgente dell'interfaccia grafica, viene clonato al commit specificato. Vengono inizializzati i sottomoduli Git e verificata la presenza della dipendenza `libevdev`.

2. **Compilazione incrociata**: la libreria LVGL e l'applicazione grafica

vengono compilate utilizzando il toolchain ARM fornito da Buildroot.

3. **Installazione**: i file binari della libreria e l'eseguibile

dell'applicazione vengono copiati nel filesystem della macchina target.

3.3.3 Il pacchetto `amel-pid`

Analogamente ai pacchetti precedenti, è stato creato il pacchetto `amel-pid-control` per la compilazione e l'installazione della libreria PID, implementata in linguaggio C++. La configurazione è riportata nel Codice sorgente 3.3.

```
AMEL_PID_VERSION = v0.0.3
AMEL_PID_SITE = git@git.amelchem.com:mpapaccioli/pid.git
AMEL_PID_SITE_METHOD = git

AMEL_PID_DEPENDENCIES = libmodbus

define AMEL_PID_BUILD_CMDS
  make -C $(@D) CC=$(TARGET_CC)
endef

define AMEL_PID_INSTALL_TARGET_CMDS
  $(INSTALL) -d $(TARGET_DIR)/opt/amel-pid/
  cp $(@D)/pid $(TARGET_DIR)/opt/amel-pid/pid
endef

$(eval $(generic-package))
```

Codice 6 — Configurazione Buildroot per il pacchetto amel-pid

3.3.4 Configurazione dei servizi di rete

Per consentire la gestione remota del dispositivo embedded, sono stati implementati i seguenti servizi di rete:

- **Server SSH**: è stato installato il server OpenSSH [11] per

permettere l'accesso remoto al sistema. È stata modificata la configurazione nel file `/etc/ssh/sshd_config` per abilitare l'autenticazione dell'utente `root` tramite password.

- **Configurazione dell'interfaccia di rete**: lo script di inizializzazione

`/etc/init.d/S50network` è stato personalizzato per configurare l'interfaccia `eth0` con un indirizzo IP statico all'avvio del sistema.

- **Rete virtuale e connettività Internet**: è stata creata una rete virtuale

che condivide la connessione Internet del PC host. Configurando il PC come default gateway del sistema embedded, è stato possibile abilitare l'accesso alla rete globale. È stato utilizzato il server DNS di Google (`8.8.8.8`) per la risoluzione dei nomi di dominio.

L'accesso a Internet è risultato necessario per la configurazione del servizio NTP (Network Time Protocol), descritto nella sezione successiva.

Al termine della configurazione di tutti i pacchetti, Buildroot assembla il filesystem e l'immagine del kernel in un file `sdcard.img`, pronto per essere scritto su una scheda SD e avviato sul dispositivo target.

3.3.5 Supporto per la porta seriale RS-232

Per abilitare la comunicazione seriale tramite la porta RS-232 presente sulla host-board Ganador, è stato necessario includere il modulo kernel `cp210x` per il controller USB-seriale Silicon Labs CP210x.

L'inclusione del modulo è stata effettuata tramite il menu di configurazione di Buildroot, selezionando il percorso:

```
Kernel modules -> USB Serial Converter support ->
USB CP210x family of UART Bridge Controllers
```

accessibile mediante il comando `make linux-menuconfig`.

3.3.6 Sincronizzazione temporale con NTP

Per garantire la corretta sincronizzazione dell'orologio di sistema, essenziale per la registrazione accurata dei dati di temperatura, è stato installato il client `ntpd` per il Network Time Protocol [12] tramite Buildroot.

3.4 Personalizzazione delle patch per Barebox e Linux

Durante la fase di sviluppo, è stata necessaria la modifica dei codici sorgenti di Barebox [10] e del kernel Linux [7] per ottimizzare le operazioni di ricompilazione frequenti.

Le personalizzazioni effettuate sono state le seguenti:

- **Kernel Linux**: è stata creata una copia del repository del kernel Linux

mantenuto da AMel. All'interno di questo repository, il modulo `cp210x` è stato abilitato di default modificando il file di configurazione `/drivers/usb/serial/Kconfig`. Questa modifica evita la necessità di riconfigurare manualmente il modulo ad ogni ricompilazione del kernel.

- **Bootloader Barebox**: è stato modificato l'ordine di ricerca dei dispositivi

all'avvio. In particolare, è stata data priorità al dispositivo `mmc2` (interfaccia SD) rispetto alla memoria `nand` (memoria Flash interna). Questa modifica consente un avvio più rapido del sistema quando il kernel è presente sulla scheda SD.

4

Installazione del sistema

4.1 Configurazione della comunicazione con l'inverter

La comunicazione con l'inverter tramite protocollo MODBUS richiede una configurazione preliminare del dispositivo. Questa operazione viene effettuata attraverso il pannello di controllo integrato, dove è necessario abilitare esplicitamente la modalità di comando remoto.

Una volta completata la configurazione dell'inverter, è possibile stabilire la comunicazione utilizzando la libreria `libmodbus`. Il procedimento si articola nei seguenti passaggi:

1. **Connessione al bus MODBUS**: stabilire il collegamento fisico e logico

con l'inverter attraverso l'interfaccia seriale configurata.

2. **Invio del comando di avvio**: scrivere nel registro della Control Word il

valore esadecimale `0x047F`, corrispondente al comando di start del motore.

L'interpretazione della documentazione tecnica fornita dal costruttore dell'inverter ha presentato alcune difficoltà iniziali. Il manuale risultava infatti poco chiaro in alcuni passaggi critici. Tuttavia, attraverso un processo iterativo di analisi e sperimentazione pratica, è stato possibile identificare correttamente la sequenza di operazioni necessarie per l'avvio del sistema.

4.2 Taratura dei parametri del controllore PID

La determinazione dei parametri ottimali per il controllore PID è stata condotta seguendo un approccio empirico, comunemente definito metodo di *tuning* manuale. Questa

metodologia prevede l'iterazione progressiva dei parametri attraverso test sperimentali direttamente sul sistema reale.

4.2.1 Fase preparatoria

Prima di procedere con la taratura vera e propria, è stata necessaria una fase di preparazione del sistema:

1. **Caricamento termico**: è stato posizionato un carico resistivo all'interno

della camera di collaudo per generare calore in modo controllato.

2. **Riscaldamento iniziale**: la temperatura interna è stata portata a

50°C, creando così le condizioni necessarie per testare il sistema di raffreddamento.

3. **Monitoraggio della temperatura**: sono stati utilizzati due sensori in

parallelo:
- Il sensore digitale DS18B20, precedentemente descritto, per l'acquisizione

dei dati utilizzati dal controllore PID
- Un termistore già presente nella camera, utilizzato come riferimento

per la validazione delle misure

Come atteso, le letture dei due sensori presentavano una leggera discrepanza, attribuibile alle diverse caratteristiche metrologiche e alle posizioni di installazione.

4.2.2 Identificazione del parametro proporzionale

La taratura è iniziata con la determinazione del guadagno proporzionale (K_p). Il procedimento seguito è stato il seguente:

1. Impostazione di un valore iniziale per il parametro proporzionale
2. Osservazione della risposta del sistema al variare del setpoint
3. Iterazione del valore fino a ottenere una convergenza verso la temperatura

desiderata

Nel caso specifico, il setpoint di riferimento è stato fissato a 40°C. Il sistema è riuscito a stabilizzarsi intorno ai 43°C, evidenziando un errore di regime non nullo caratteristico di un controllo esclusivamente proporzionale.

È importante sottolineare che, trattandosi di un sistema di raffreddamento, i parametri del controllore assumono valori negativi. In particolare, il valore identificato per il parametro proporzionale è risultato essere $K_p = -3000$.

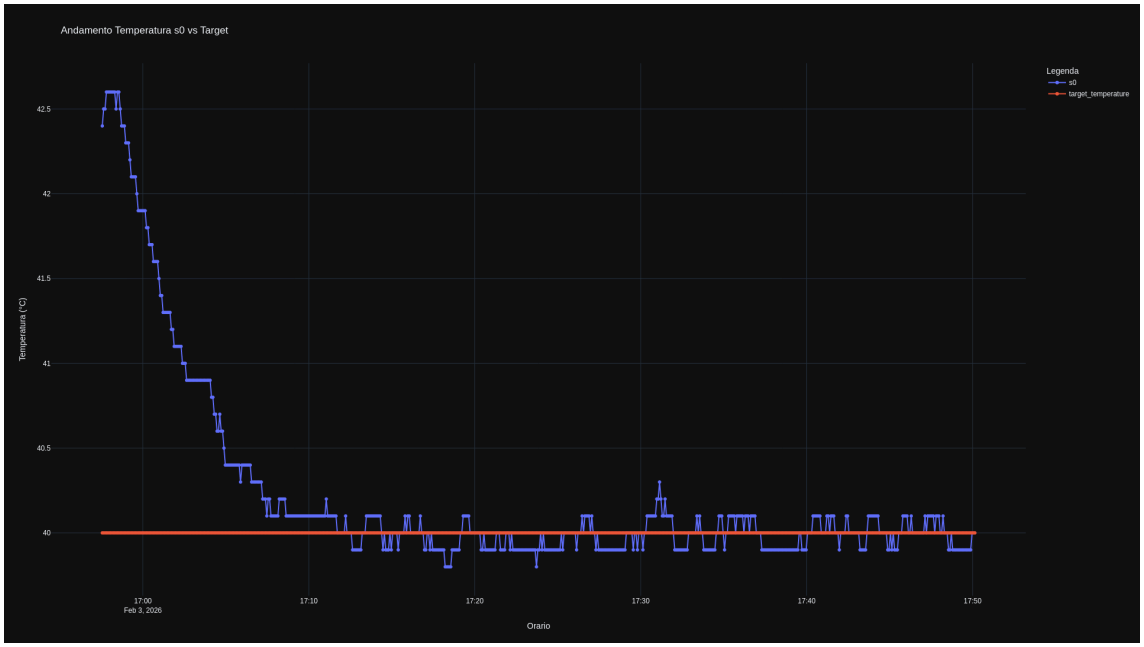

Figura 3 — Andamento della temperatura durante la fase di taratura del controllore PID

4.2.3 Integrazione della componente integrale

Successivamente, è stato introdotto il parametro integrale (K_i) per eliminare l'errore di regime osservato nella fase precedente. L'integrazione di questo parametro consente al sistema di raggiungere asintoticamente il setpoint desiderato.

Il procedimento di taratura ha previsto:

1. Aggiunta graduale del termine integrale a partire da valori ridotti
2. Monitoraggio continuo della risposta per evitare fenomeni di

sovraelongazione eccessiva o instabilità

3. Ottimizzazione finale del parametro fino al raggiungimento di un compromesso

tra velocità di risposta e assenza di oscillazioni

Il valore finale identificato per il parametro integrale è risultato essere $K_i = -15$. Si evidenzia la significativa differenza di magnitudine tra i due parametri: il guadagno proporzionale è circa duecento volte superiore a quello integrale. Questa caratteristica è tipica dei sistemi di controllo termico, dove la costante di tempo del processo è elevata e richiede un'azione integrale moderata per evitare instabilità.

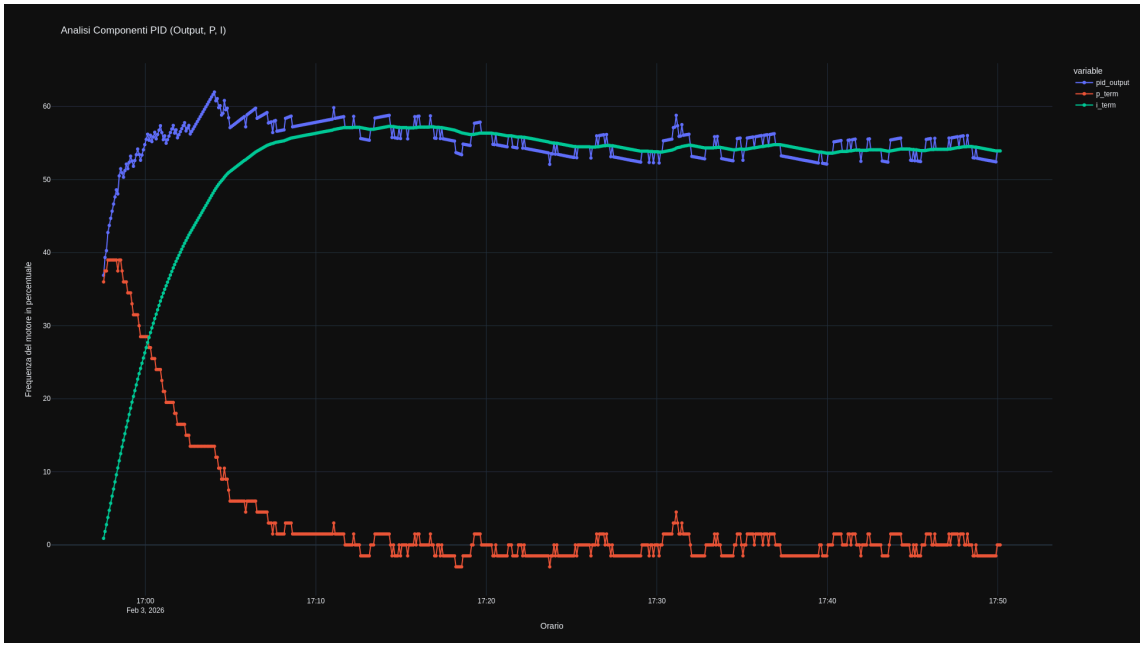

Figura 4 — Contributo dei singoli termini del controllore PID: componente proporzionale (in alto) e componente integrale (in basso)

Il secondo grafico illustra chiaramente il comportamento del controllore: quando il sistema si avvicina al setpoint, il contributo proporzionale tende a zero, mentre il termine integrale assume il ruolo principale nel mantenere la temperatura stabile attraverso una correzione continua dell'errore cumulato nel tempo.

4.3 Risoluzione dei problemi riscontrati

Durante la fase di installazione e messa in servizio del sistema, sono emerse diverse problematiche non previste nella fase di progettazione. Di seguito vengono documentate le principali anomalie riscontrate e le relative soluzioni implementate.

4.3.1 Gestione della concorrenza nel framework LVGL

La prima problematica significativa è emersa nel modulo di visualizzazione `temp-control`, sviluppato con il framework LVGL. Inizialmente, l'aggiornamento della temperatura visualizzata sul display era gestito attraverso un meccanismo basato su segnali UNIX: il processo `pid-control` inviava un segnale `SIGUSR1` al processo `temp-control` per notificare la disponibilità di un nuovo valore di temperatura.

Tuttavia, questa implementazione ha generato un errore di runtime dovuto alla natura single-threaded della libreria LVGL. Il framework non supporta infatti l'aggiornamento di elementi grafici da parte di signal handler, in quanto tali callback vengono eseguiti in un contesto asincrono che può interferire con il ciclo principale di rendering.

La soluzione adottata ha previsto la sostituzione del meccanismo basato su segnali con un approccio basato su timer. In particolare, è stata utilizzata la funzione `lv_timer_create` fornita dalla libreria LVGL, che consente di schedulare callback all'interno del contesto di esecuzione principale del framework.

```
commit cc35bb1af8959f3dfb39b2dcc8c0a98021615001
Author: Mattia Papaccioli <mattiapapaccioli@gmail.com>
Date:   Fri Feb 6 12:04:58 2026 +0100

    fix: update labels with timer to avoid runtime error

diff --git a/src/main.c b/src/main.c
index 616e062..fcc48e4 100644
--- a/src/main.c
+++ b/src/main.c
@@ -154,6 +154,10 @@ const int padding_button = 50;
 const int height_button  = 50;
 const int width_button   = 50;

+static void update_current_temperature_cb(lv_timer_t* timer) {
+    update_current_temperature();
+}
+
 lv_obj_t * create_card(lv_obj_t * parent, const char * header_text, lv_obj_t
 ** value_label_ptr, lv_color_t bg_color,
                        const lv_img_dsc_t * icon_dsc)
@@ -341,11 +345,7 @@ int main(int argc, char ** argv)
     lv_img_set_src(decrement_icon, &minus);
     lv_obj_center(decrement_icon);

-    struct sigaction sa = {0};
-    sa.sa_handler = update_current_temperature;
-    sa.sa_flags   = SA_RESTART;
-    sigaction(SIGUSR1, &sa, NULL);
-    sigprocmask(SIG_UNBLOCK, &set, NULL);
+    lv_timer_create(update_current_temperature_cb, 1000, NULL);

     /* Enter the run loop of the selected backend */
     driver_backends_run_loop();
```

Codice 7 — Commit di correzione: sostituzione del signal handler con timer LVGL (`git show cc35bb1`)

Questa modifica garantisce che l'aggiornamento dell'interfaccia grafica avvenga in modo sicuro e sincronizzato con il ciclo principale di LVGL, eliminando il rischio di race condition e accessi concorrenti non protetti.

4.3.2 Gestione delle risorse nella comunicazione MODBUS

Una seconda problematica critica è stata identificata nel modulo `pid-control`, specificatamente nella gestione della connessione MODBUS con l'inverter. L'implementazione originale prevedeva l'apertura di una nuova connessione attraverso la

funzione `modbus_connect` ogni volta che era necessario aggiornare la velocità della ventola.

Questo approccio, apparentemente innocuo, ha generato un errore grave nel sistema: dopo circa 1024 iterazioni del ciclo di controllo, l'applicazione terminava in modo anomalo. L'analisi del problema ha evidenziato che ogni chiamata a `modbus_connect` allocava un nuovo file descriptor senza che questi venissero opportunamente rilasciati, causando l'esaurimento delle risorse disponibili (limite imposto dal parametro `ulimit` del sistema operativo).

La causa principale risiedeva nella mancata chiusura della connessione dopo l'invio dei comandi. Nel tempo, il numero di file descriptor aperti cresceva monotonicamente fino a raggiungere il limite massimo consentito dal kernel.

La soluzione implementata ha previsto una ristrutturazione significativa dell'architettura di comunicazione:

1. **Connessione persistente**: la connessione MODBUS viene stabilita una sola

volta durante l'inizializzazione del modulo `pid-control`, all'interno della funzione `main`

2. **Riutilizzo del contesto**: il puntatore `modbus_t *ctx` viene passato

alle funzioni di libreria che necessitano di comunicare con l'inverter, evitando di creare nuove connessioni

3. **Rilascio delle risorse**: la connessione viene chiusa in modo esplicito

solo al termine dell'esecuzione del programma

```
commit e8f11fc2cf56682c62a2d3c52088442434524a92
Author: Mattia Papaccioli <mattiapapaccioli@gmail.com>
Date:   Fri Feb 6 17:10:30 2026 +0100

    fix: open file descriptor once

    bug fix that caused too many open files.
    the file was open every time we sent a command to the inverter.
    only one time is necessary.
    dont trust blindly llm code ;).

diff --git a/src/lib/devices/acs_310_modbus.c
b/src/lib/devices/acs_310_modbus.c
index 863b4de..869ff01 100644
--- a/src/lib/devices/acs_310_modbus.c
+++ b/src/lib/devices/acs_310_modbus.c
@@ -84,11 +84,7 @@ void check_faults(modbus_t *ctx) {
 bool send_command(modbus_t *ctx, int register_addr, uint16_t command) {
     if (!ctx) return false;

-    if (modbus_connect(ctx) == -1) {
-        LOG_ERROR("Connection failed: %s\\n", modbus_strerror(errno));
-        modbus_free(ctx);
-        return false;
-    }
+

     int rc = modbus_write_register(ctx, register_addr, command);
     if (rc == -1) {
diff --git a/src/main.c b/src/main.c
index 08e2bf7..740dc78 100644
--- a/src/main.c
+++ b/src/main.c
@@ -14,6 +14,7 @@
 #include <sys/timerfd.h>
 #include <sched.h>
 #include <sys/stat.h>
+#include <errno.h>

 #include <common-control/common-control.h>
 #include <common-control/config.h>
@@ -225,6 +226,12 @@ int main()

     modbus_t *ctx = get_client();

+    if (modbus_connect(ctx) == -1) {
+        LOG_ERROR("Connection failed: %s\\n", modbus_strerror(errno));
+        modbus_free(ctx);
+        return false;
+    }
+
     log_init();
     log_set_level(LOG_DEBUG);
     log_set_output(LOG_OUTPUT_CONSOLE);
```

Codice 8 — Commit di correzione: gestione ottimizzata dei file descriptor (`git show f0bbc5`)

Questa modifica architetturale ha non solo risolto il problema del crash del sistema, ma ha inoltre migliorato le prestazioni complessive riducendo l'overhead associato alla creazione ripetuta di connessioni di rete.

5

Manuale utente

Il presente capitolo fornisce le istruzioni dettagliate per la configurazione e l'utilizzo del sistema di controllo temperatura. Tale manuale è strutturato in quattro sezioni principali: la compilazione del sistema operativo mediante Buildroot, la preparazione della scheda di memoria SD, la configurazione del dispositivo embedded e, infine, le procedure di inizializzazione e avvio del sistema.

5.1 Compilazione del sistema operativo tramite Buildroot

La prima fase del processo di configurazione richiede la compilazione del sistema operativo embedded. Questa operazione viene effettuata mediante Buildroot, un framework open-source per la generazione di sistemi Linux embedded.

5.1.1 Procedura di clonazione del repository

Inizialmente, è necessario ottenere il codice sorgente del progetto. Il repository Git è ospitato presso il server dell'azienda ed è accessibile tramite due modalità:

- **Modalità SSH** (consigliata per gli sviluppatori con chiavi configurate): `git clone git@git.amelchem.com:amel/buildroot.git`
- **Modalità HTTPS** (alternativa generale): `git clone https://git.amelchem.com/amel/buildroot.git`

Una volta completata la clonazione, è fondamentale posizionarsi sul branch corretto denominato `Vulcano-papaccioli`. Questo branch contiene la configurazione specifica per la board utilizzata nel progetto.

5.1.2 Processo di compilazione

Dopo aver effettuato il checkout sul branch corretto, si può avviare il processo di compilazione mediante il comando `make`. Tale comando eseguirà automaticamente la compilazione di tutti i componenti necessari, tra cui:

- Il kernel Linux ottimizzato per l'architettura target
- Il bootloader Barebox per la gestione dell'avvio
- Tutti i pacchetti e le dipendenze richieste dal sistema

Considerazioni importanti sulla compilazione:

1. **Durata:** la prima compilazione richiede circa un'ora, poiché Buildroot deve scaricare e compilare tutti i pacchetti dalla fonte.
2. **Spazio su disco:** sono necessari almeno 20 GB di spazio libero sul disco rigido per ospitare i sorgenti scaricati, i file oggetto intermedi e le immagini finali del sistema.
3. **Connessione di rete:** durante il processo è richiesta una connessione Internet stabile per il download dei pacchetti.

5.2 Preparazione della scheda di memoria SD

Al termine della compilazione, Buildroot genera nella directory `output` tutti i file necessari per il deployment del sistema. In particolare, il file `output/images/sdcard.img` rappresenta l'immagine completa del sistema operativo pronta per essere installata sulla scheda di memoria.

5.2.1 Procedura di scrittura dell'immagine

Per trasferire l'immagine sulla scheda SD è necessario utilizzare il comando `dd`, che effettua una copia bit-per-bit del file. Tale approccio garantisce la preservazione esatta di tutte le partizioni e i filesystem presenti nell'immagine.

Il comando da eseguire è il seguente:

```
sudo dd if=output/images/sdcard.img of=/dev/mmcblk0 bs=4M status=progress
conv=fsync
```

Dove:
- `if` (input file) specifica il percorso dell'immagine sorgente
- `of` (output file) indica il dispositivo di destinazione (la scheda SD)
- `bs=4M` imposta la dimensione del blocco a 4 megabyte per ottimizzare la velocità di trasferimento
- `status=progress` visualizza l'avanzamento dell'operazione
- `conv=fsync` garantisce la sincronizzazione dei dati sul dispositivo prima della conclusione del comando

Nota: è necessario adattare i percorsi (`if`) e il nome del dispositivo (`of`) in base alla configurazione specifica della macchina utilizzata. Per identificare correttamente il dispositivo della scheda SD, si consiglia di utilizzare il comando `lsblk` prima e dopo l'inserimento della scheda.

5.3 Configurazione del dispositivo embedded

Una volta preparata la scheda di memoria, è possibile procedere con la configurazione del dispositivo target.

5.3.1 Avvio del sistema

La procedura di avvio si articola nei seguenti passaggi:

1. **Inserimento della scheda:** estrarre con cautela la scheda SD dal computer e inserirla nello slot apposito presente sulla board Ganador.
2. **Alimentazione della board:** collegare l'alimentazione alla board per avviare il processo di boot.
3. **Attesa dell'avvio:** il sistema impiega circa un minuto per completare l'avvio. Durante questa fase vengono eseguite le seguenti operazioni:
   - Inizializzazione del kernel Linux
   - Montaggio dei filesystem
   - Generazione delle chiavi SSH (operazione che richiede circa 30 secondi)

Al termine di questo processo, il sistema è pronto per essere configurato e utilizzato.

5.3.2 Modalità di connessione al dispositivo

Sono disponibili due modalità alternative per accedere al sistema embedded:

Connessione tramite porta seriale

La connessione seriale rappresenta il metodo più affidabile per l'accesso iniziale al sistema, poiché non richiede la configurazione di rete.

Requisiti hardware:
- Adattatore USB-seriale compatibile con livelli logici RS-232
- Cavi di collegamento per i pin UART della board

Configurazione software: Per la comunicazione seriale è necessario utilizzare un emulatore di terminale. Il software consigliato è `minicom`, disponibile sulla maggior parte delle distribuzioni Linux.

Procedura di connessione:

1. Collegare l'adattatore USB-seriale al computer host
2. Connettere i pin dell'adattatore alla board Ganador secondo lo schema riportato nella Figura 5
3. Identificare il nome del device seriale (tipicamente `/dev/ttyUSB0` su sistemi Linux) mediante il comando `dmesg | grep tty`
4. Avviare minicom specificando il device: `minicom -D /dev/ttyUSB0`

Figura 5 — Schema di collegamento dei pin per la comunicazione seriale

Connessione tramite SSH

La connessione SSH offre un accesso remoto più flessibile, particolarmente utile quando il dispositivo è già configurato e integrato nell'ambiente operativo.

Per stabilire la connessione, utilizzare il comando:

```
ssh root@10.42.0.2
```

Le credenziali di accesso predefinite sono:
- **Username:** `root`
- **Password:** `root`

Personalizzazione delle credenziali e della rete:

Qualora fosse necessario modificare la password di root o la configurazione di rete prima della compilazione, è possibile intervenire direttamente sulla configurazione di Buildroot.

La Codice 9 mostra la modifica della password di root nel file di configurazione `vulcanoa5_defconfig`, mentre la Codice 10 illustra la configurazione dell'interfaccia di rete.

```
commit fc474df54681e9057202aac05b035762825c07f4
Author: Mattia Papaccioli <mattiapapaccioli@gmail.com>
Date:   Mon Dec 8 14:39:56 2025 +0100

    vulcanoa5_defconfig: change root password

diff --git a/configs/vulcanoa5_defconfig b/configs/vulcanoa5_defconfig
index 7f9b3db1c1..2b66c41777 100644
--- a/configs/vulcanoa5_defconfig
+++ b/configs/vulcanoa5_defconfig
@@ -11,7 +11,7 @@ BR2_CCACHE_INITIAL_SETUP="--max-size=10G"
 BR2_TARGET_GENERIC_HOSTNAME="vulcanoa5"
 BR2_TARGET_GENERIC_PASSWD_SHA512=y
 BR2_ROOTFS_DEVICE_CREATION_DYNAMIC_EUDEV=y
-BR2_TARGET_GENERIC_ROOT_PASSWD="$$6$$IzRpYk0B$$oBTolwsRP7IOx4jxgDuS9lv6NoFt5G/fODQqCVtP/OmJ5L25vcT8H62mWr.gxSihpwTLM.2nXn1zl.tTZyqXN0"
+BR2_TARGET_GENERIC_ROOT_PASSWD="root"
 BR2_ROOTFS_OVERLAY="board/amel/vulcano/fs-overlay"
 BR2_ROOTFS_POST_BUILD_SCRIPT="board/amel/vulcano/post-build-script"
 BR2_ROOTFS_POST_IMAGE_SCRIPT="board/amel/vulcano/post-image-script"
@@ -39,6 +39,7 @@ BR2_PACKAGE_OPENSSH_SANDBOX=y

 BR2_PACKAGE_AMEL_TEMP_CONTROL=y
 BR2_PACKAGE_AMEL_PID=y
+
 BR2_PACKAGE_XORG7=y
 BR2_PACKAGE_XSERVER_XORG_SERVER=y
 BR2_PACKAGE_XAPP_XINIT=y
```

Codice 9 — Modifica della password di root in Buildroot

```
commit 7f0ab83b26eca8b584839a693734d12126465cdc
Author: Mattia Papaccioli <92176188+sb0ogway@users.noreply.github.com>
Date:   Tue Dec 23 19:55:32 2025 +0100

    package/ifupdown-scripts: adding interface configuration to connect
    to internet

diff --git a/package/ifupdown-scripts/S40network
b/package/ifupdown-scripts/S40network
index 7d23a89120..642c5013ac 100644
--- a/package/ifupdown-scripts/S40network
+++ b/package/ifupdown-scripts/S40network
@@ -26,8 +26,5 @@ case "$1" in
  exit 1
  esac

-ip a a 192.168.8.11/24 dev eth0
-ip link set eth0 up
-
  exit $?

 diff --git a/package/ifupdown-scripts/ifupdown-scripts.mk
b/package/ifupdown-scripts/ifupdown-scripts.mk
index 5ef032142c..7397e2d034 100644
--- a/package/ifupdown-scripts/ifupdown-scripts.mk
+++ b/package/ifupdown-scripts/ifupdown-scripts.mk
@@ -14,6 +14,13 @@ define IFUPDOWN_SCRIPTS_LOCALHOST
    echo ; \
    echo "auto lo"; \
    echo "iface lo inet loopback"; \
+   echo ; \
+   echo "auto eth0";  \
+   echo "iface eth0 inet static"; \
+   echo "    address 10.42.0.2"; \
+   echo "    netmask 255.255.255.0"; \
+   echo "    post-up ip route add default via 10.42.0.1"; \
+   echo "    post-up echo \"nameserver 8.8.8.8\" >
/etc/resolv.conf"; \
  ) >> $(TARGET_DIR)/etc/network/interfaces
  endef
```

Codice 10 — Configurazione dell'interfaccia di rete in Buildroot

5.4 Inizializzazione e avvio del sistema

Dopo aver stabilito la connessione con il dispositivo, è necessario procedere con la configurazione del software applicativo e l'avvio dei servizi.

5.4.1 Configurazione dell'ambiente

La directory di lavoro principale del sistema è `/opt/amel/`. È necessario posizionarsi in tale directory per accedere agli script e ai file di configurazione:

```
cd /opt/amel/
```

All'interno di questa directory è presente il file `config.env`, che contiene le variabili d'ambiente necessarie per la personalizzazione del comportamento del sistema. Tale file deve essere modificato in base alle esigenze specifiche dell'installazione.

5.4.2 Calibrazione del touchscreen

Il sistema utilizza la libreria grafica LVGL per l'interfaccia utente. Per garantire il corretto funzionamento del touchscreen è necessario effettuare una procedura di calibrazione.

La calibrazione richiede la determinazione dei seguenti parametri, da inserire nel file `config.env`:

- `LV_MIN_X`: valore minimo sull'asse X
- `LV_MIN_Y`: valore minimo sull'asse Y
- `LV_MAX_X`: valore massimo sull'asse X
- `LV_MAX_Y`: valore massimo sull'asse Y

Procedura di calibrazione:

1. Eseguire il comando `evtest /dev/input/event0` per monitorare gli eventi di input del touchscreen
2. Toccare successivamente i quattro angoli dello schermo
3. Annotare i valori minimi e massimi rilevati per ciascun asse
4. Inserire tali valori nel file `config.env`

5.4.3 Script di gestione del sistema

Il sistema fornisce tre script principali per la gestione dei servizi:

1. Script di inizializzazione (`init.sh`)

Questo script deve essere eseguito una sola volta, tipicamente dopo la prima installazione o dopo una reimpostazione del sistema. Il suo compito è creare la struttura di directory e i file necessari al funzionamento dell'applicazione.

```
./init.sh
```

2. Script di avvio (`run.sh`)

Per avviare il sistema di controllo temperatura è sufficiente eseguire lo script `run.sh`. Tale script si occupa di avviare in sequenza tutti i processi necessari, inclusi il servizio di acquisizione dati, il controllore PID e l'interfaccia grafica.

```
./run.sh
```

3. Script di arresto (`kill.sh`)

Per terminare correttamente l'esecuzione del sistema è necessario utilizzare lo script `kill.sh`. Questo script provvede a interrompere in modo ordinato tutti i processi avviati dallo script `run.sh`, garantendo la chiusura corretta delle risorse e il salvataggio dei dati.

```
./kill.sh
```

5.4.4 Monitoraggio e logging

Il sistema mantiene una traccia dettagliata delle operazioni mediante file di log, archiviati nella directory `/opt/amel/logs/`.

Tali log contengono:
- Messaggi di avvio e arresto dei processi
- Eventuali errori e warning
- Informazioni di debug relative al funzionamento del sistema

La consultazione dei log è fondamentale per il monitoraggio del sistema e per l'individuazione di eventuali malfunzionamenti. È possibile visualizzare i log in tempo reale mediante il comando `tail -f` oppure analizzarli a posteriori con strumenti come `less` o `grep`.

6

Conclusione

6.1 L'importanza del testing integrato sul campo

L'esperienza maturata durante la fase di installazione e collaudo del sistema ha evidenziato un aspetto fondamentale dello sviluppo embedded: la necessità di effettuare prove di integrazione direttamente in ambiente operativo.

Questa considerazione si articola in tre punti chiave:

1. **Limiti dell'unit testing**: le tecniche di testing automatico su

singoli moduli, sebbene indispensabili, non sono sufficienti a garantire il corretto funzionamento dell'intero sistema. Alcuni malfunzionamenti emergono esclusivamente quando tutti i componenti operano in modo coordinato.

2. **Problematiche di integrazione**: le anomalie riscontrate durante

l'installazione hanno dimostrato come errori di comunicazione tra dispositivi, interferenze elettromagnetiche o incompatibilità temporali non siano riproducibili in ambiente di sviluppo isolato.

3. **Validazione completa**: solo attraverso test eseguiti con l'intera

infrastruttura operativa è stato possibile verificare il comportamento reale del sistema, identificando criticità che altrimenti sarebbero rimaste occulte fino alla messa in produzione.

Questa esperienza conferma la rilevanza delle metodologie di integration testing e system testing nell'ambito dell'ingegneria dei sistemi embedded, oltre alle pratiche di unit testing tradizionali.

6.2 Competenze acquisite e riflessioni professionali

Il presente progetto di tesi ha costituito un'occasione significativa di crescita professionale e personale, permettendo l'acquisizione di competenze trasversali di rilevante interesse per il percorso di formazione ingegneristica.

6.2.1 Strumenti di sviluppo collaborativo

L'approfondimento del sistema di versionamento `git` ha rappresentato un elemento formativo di particolare valore. L'apprendimento delle pratiche di branching, merging e gestione delle release costituisce una competenza fondamentale per l'attività professionale nel settore dello sviluppo software, favorendo il lavoro in team e la tracciabilità delle modifiche.

6.2.2 Architetture embedded

Lo studio approfondito dei microcontrollori, con particolare riferimento alla famiglia ESP32, ha permesso di acquisire conoscenze relative a:
- La programmazione a basso livello e la gestione delle risorse hardware

limitate
- L'interfacciamento con periferiche esterne attraverso bus di comunicazione

standard
- L'ottimizzazione del codice per sistemi con vincoli temporali real-time

Questo settore ha suscitato notevole interesse, aprendo prospettive di approfondimento futuro nel campo dell'Internet of Things e dei sistemi cyber-fisici.

6.2.3 Interdisciplinarità ingegneristica

Il progetto ha inoltre consentito di esplorare aspetti dell'ingegneria elettrica, disciplina complementare ma distinta dall'informatica. In particolare, si è acquisita consapevolezza delle problematiche relative a:
- La gestione dei carichi elettrici in ambiente industriale
- Le tecniche di protezione dei circuiti
- Le interferenze elettromagnetiche e le strategie di schermatura

Questa dimensione interdisciplinare si è rivelata essenziale per comprendere l'interazione tra il software embedded e l'hardware fisico su cui opera.

6.2.4 Applicazione di protocolli industriali

L'implementazione pratica del protocollo Modbus ha permesso di sperimentare direttamente le dinamiche della comunicazione industriale standardizzata. La possibilità di

controllare attuatori di potenza, come ventilatori industriali, attraverso comandi software ha rappresentato un risultato concreto e soddisfacente, confermando l'efficacia delle soluzioni progettate.

6.3 Ringraziamenti

Desidero esprimere la mia gratitudine ad AMEL s.r.l. per avermi offerto l'opportunità di sviluppare questo progetto in un contesto aziendale stimolante e professionale.

Un ringraziamento particolare va a Edoardo [Cognome] per il prezioso supporto tecnico e la collaborazione costante durante tutte le fasi di sviluppo, dalla progettazione alla messa in opera. Estendo i miei ringraziamenti a tutti i colleghi dell'azienda per l'accoglienza ricevuta e per l'ambiente di lavoro favorevole allo scambio di conoscenze.

Vorrei inoltre ringraziare i docenti del Corso di Laurea in Ingegneria Informatica, in particolare il Professor [Nome] Dossi per la supervisione accademica e la disponibilità dimostrata durante il percorso di tesi. Un sentito grazie anche ai compagni di corso, con cui le discussioni e il confronto hanno arricchito significativamente la mia esperienza universitaria.

Infine, dedico un ringraziamento speciale alla mia famiglia per il sostegno incondizionato e la pazienza dimostrata durante l'intero percorso di studi, nonostante le difficoltà e le sfide affrontate.

A Riferimenti

[1] S. Raimbault, «libmodbus - A Modbus library for Linux, Mac OS X, FreeBSD, QNX and Windows». [Online]. Disponibile su: https://github.com/stephane/libmodbus

[2] L. Contributors, «LVGL». [Online]. Disponibile su: https://github.com/lvgl/lvgl

[3] M. Integrated, «DS18B20 Programmable Resolution 1-Wire Digital Thermometer». [Online]. Disponibile su: https://www.maximintegrated.com/en/products/sensors/DS18B20.html

[4] L. Contributors, «Linux template for LVGL». [Online]. Disponibile su: https://github.com/lvgl/lv_port_linux

[5] [Online]. Disponibile su: https://www.freedesktop.org/wiki/Software/libevdev/

[6] K. Inc., «CMake - Cross Platform Make». [Online]. Disponibile su: https://cmake.org/

[7] L. Torvalds e L. kernel developers, «The Linux Kernel Archives». [Online]. Disponibile su: https://www.kernel.org/

[8] B. contributors, [Online]. Disponibile su: https://buildroot.org/

[9] M. T. Inc., «AT91Bootstrap». [Online]. Disponibile su: https://github.com/linux4sam/at91bootstrap

[10] B. developers, «Barebox - bootloader for embedded systems». [Online]. Disponibile su: https://barebox.org/

[11] O. developers, «OpenSSH». [Online]. Disponibile su: https://www.openssh.com/

[12] N. T. Foundation, «Network Time Protocol - NTP». [Online]. Disponibile su: https://www.ntp.org/